

MÓDULO I

Configuração do Ambiente e Conceitos Básicos em PHP

MÓDULO I

O que é PHP e para que serve?

O PHP (acrônimo recursivo para **PHP: Hypertext Preprocessor** ou, em tradução, *Pré-Processador de Hipertexto*) já foi muito chamado de linguagem de **script**, mas podemos entendê-lo como uma linguagem de programação interpretada. Isso significa que, quando alteramos o nosso código, não precisamos compilá-lo novamente para que ele seja legível por algum programa ou pela própria máquina, o que traz algumas vantagens e desvantagens - temos como vantagem principalmente a produtividade.

Muitos consideram o **PHP** uma linguagem para a **Web**, mas não é exatamente assim. O **PHP** funciona em outros ambientes, como: **IOT** (*Internet of Things*, ou "*Internet das Coisas*"), na linha de comando, pode criar aplicativos desktop, entre outras possibilidades. Apesar disso, o ambiente mais comum, e onde ele se comporta melhor, realmente é na **Web** - instalado em servidores **Web** e servindo páginas com conteúdo dinâmico.

MÓDULO I

Como qualquer linguagem que aprendemos na faculdade ou nos livros, nós começamos a estudar o **PHP** executando comandos no terminal - ou seja, na linha de comando.

Inclusive é possível criar scripts muito poderosos na linha de comando.

O **PHP** foi criado em **1994** e, desde então, teve uma grande evolução. A cada versão foram adicionadas novas funcionalidades responsáveis por tornar a linguagem ainda mais popular e poderosa.

Atualmente, o **PHP** está na versão **8.1.4**.

MÓDULO I

HISTÓRIA DO PHP

Por volta de **1994**, o programador dinamarquês **Rasmus Lerdorf**, utilizava um conjunto de códigos binários Common Gateway Interface (**CGI**), escritos em **linguagem C**. Esse conjunto tinha o objetivo de fazer a ligação lógica entre dois sistemas ou servidores pela internet.

Dessa forma, esse conjunto de códigos, que nada mais eram do que um aglomerado de scripts, foi inicialmente nomeado como **PHP/Fi**. Esta seria uma versão prematura do **PHP**. Com ele, **Rasmus** era capaz de monitorar o tráfego de acessos ao seu site particular.

MÓDULO I

Foto: Rasmus Lerdorf



Com o passar do tempo, **Rasmus** foi desenvolvendo novos **scripts** para incluir no seu site. Ou seja, esse pacote de **Scripts**, foi ganhando mais recursos e implementações. **Rasmus Lerdorf** renomeou seu produto para **Personal Home Page Tools**, ou **PHP Tools**. Portanto, o **PHP** passou a ser um conjunto de ferramentas úteis para integração entre sistemas e desenvolvimento de aplicações web sem muitas dificuldades.

MÓDULO I

Em **1995**, **Rasmus** então liberou o código fonte do **PHP Tools** para o público geral.

Dessa forma, abriu-se as portas para que ideias de outros desenvolvedores pudessem mexer e otimizar a estrutura do código-fonte livremente. Foi assim que a linguagem foi se aperfeiçoando.

Dessa forma, o **PHP Tools** foi ganhando cada vez mais popularidade. A linguagem de programação ficou mundialmente conhecida como **Personal Home Page (PHP)** ainda em **1995**.

Mais tarde, em **1997**, recebeu uma atualização em seu nome, passando a ser **PHP Hypertext Preprocessor**.

MÓDULO I

CARACTERÍSTICAS DO PHP

A linguagem **PHP** é muito utilizada pelos sites ao redor do mundo.

De acordo com estatísticas apresentadas pelo **W3Techs**, o **PHP** é utilizado por quase **80%** dos sites da internet. A razão de toda essa popularidade não poderia ser outra: **as características da linguagem**.

Configura algumas das principais:

1. Fácil aprendizagem:

Uma das principais características do **PHP** é que ela é de fácil aprendizagem, a sintaxe utilizada no **PHP** é simples e fácil de aprender, pois é similar a outras existentes no mercado. Através da própria **documentação oficial** você já é capaz de aprender muita coisa. Portanto, a linguagem é uma das mais acessíveis para aprender a usar.

MÓDULO I

2. Código Aberto:

A linguagem **PHP** possui código aberto, ou seja, é gratuita para qualquer usuário. Além disso, permite que desenvolvedores e programadores experientes tenham acesso e possam fazer alterações e atualizações periódicas no seu código fonte. Dessa forma, o **PHP** está sempre ganhando recursos e evoluindo constantemente.

3. Possui alto desempenho:

O **PHP** é uma linguagem de alto desempenho, capaz de suportar grandes quantidades de dados. Ela consegue executar muitas funções e consumir muitos dados com a utilização de poucos recursos do servidor, assim não compromete o **desempenho** e a **velocidade** do **servidor** que está executando a aplicação.

MÓDULO I

4. Compatibilidade com Banco de dados:

Além de possuir um bom desempenho, é importante que as linguagens também tenham um bom nível de compatibilidade com bancos de dados, já que eles são parte fundamental da estrutura de sites.

A linguagem **PHP** é compatível com os principais bancos de dados, como **MySQL**, **SQLite**, **Firebird**, **Interbase** e **Oracle**.

5. É multiplataforma:

Isso significa que nós podemos utilizar e rodar a linguagem **PHP** numa variedade de sistemas operacionais. **Windows**, **Linux** (*e suas distribuições*) e **MacOS** são alguns deles.

MÓDULO I

6. É a linguagem de programação do WordPress:

O **WordPress** é o **CMS**(*Content Management System* ou, em tradução, *Sistema de Gerenciamento de Conteúdo*) mais procurado e utilizado no mundo. Ele é baseado na linguagem **PHP** e toda sua programação é feita com ele. Portanto, para ser um desenvolvedor **WordPress**, é obrigatório ter conhecimento sobre a linguagem **PHP**, o que aumenta cada vez mais a comunidade do **PHP**, acarretando diversos benefícios para a evolução do mercado e do **PHP**.

7. Possui uma grande quantidade de framework e bibliotecas

Frameworks e **bibliotecas** facilitam muito a vida de um desenvolvedor, possibilitando realizar tarefas em menos tempo e com bem menos linhas de código. O **PHP** possui muitos **frameworks** e **bibliotecas**, o que não faltam são opções para utilizar. Como exemplos de **Frameworks** conhecidos temos o **Laravel**, o **CodeIgniter**, o **Zend** e o **Symfony**.

MÓDULO I

Lado do Servidor (backend) e Lado do Cliente (frontend)

Linguagens de script podem rodar tanto do lado do **servidor (backend)** quanto do lado do **cliente (frontend)**.

Scripts do tipo **frontend** são processados pelos **navegadores**. Quando o seu **browser** – também conhecido como o **cliente** – solicita uma página contendo scripts do lado do **cliente**, o servidor responde enviando os códigos-fontes que são executáveis pelo **navegador**.

Por outro lado, linguagem de scripts do tipo **backend** significam que esses scripts são executados nos **servidores** antes de serem enviados ao **navegador**. Então ao invés de mandar o código-fonte, os **servidores** da web processam (*parse*) os códigos antes ao transformá-los num formato **HTML** puro.

MÓDULO I

Quem usa PHP?

Várias empresas estabelecidas e gigantes da tecnologia utilizam **PHP** para executar seus servidores e disponibilizar seus serviços, alguns exemplos são:

- **Facebook:** Facebook utiliza **PHP** para alimentar seu site. Por sua vez, a empresa contribuiu com a comunidade criando uma implementação conhecida como **Hip Hop** para **PHP**.
- **Wikipedia:** uma das maiores fontes de informação colaborativa do mundo sobre qualquer assunto, a Wikipedia é construída em **PHP**.
- **Sistemas de gerenciamento de conteúdo (CMS):** o sistema de gerenciamento de conteúdo mais popular do mundo, o **WordPress**, é construído em **PHP**. Outros sistemas de gerenciamento de conteúdo, como **Drupal**, **Joomla** e **Magento**, também são construídos em **PHP**. **Shopify** também roda em **PHP**.

MÓDULO I

Onde é usado o PHP e o que dá pra fazer com ele?

O **PHP** pode ser utilizado para desenvolver diversas coisas incríveis, confira algumas delas:

1. Desenvolvimento de sites dinâmicos

Sites dinâmicos são aqueles em que os elementos mostrados neles não ficam estáticos. Ou seja, suas páginas são gerenciadas por uma aplicação hospedada no servidor, um sistema do lado do servidor (*Back End*). Através do **PHP**, podemos determinar como uma página será mostrada quando for carregada, podendo manipular o conteúdo que será exibido.

MÓDULO I

2. Desenvolvimento de aplicações para a web

O **PHP** permite criar aplicações de todos os tipos para a web. Podemos criar um fórum de dúvidas ou ainda criar uma plataforma de cursos, como a Alura. Podemos também incluir formulários de contato em nossas páginas e receber dados através deles. Tudo isso é possível utilizando o **PHP**.

3. Criação de plugins e temas para WordPress

Essa é uma das grandes utilidades do **PHP** nos dias de hoje. O **PHP** pode desenvolver **plugins** e **temas** e, inclusive, pode vender essas criações através da plataforma. No **WordPress**, o **CMS** (*Sistema de Gerenciamento de Conteúdo*) mais utilizado no mundo, a procura por **plugins** e **temas** é alta, e por este motivo muitos desenvolvedores ou pessoas que estão iniciando na programação optam por aprender **PHP**.

MÓDULO I

4. Criação de scripts de linha de comando

Com **PHP** é possível executar scripts diretamente do terminal, utilizando a linha de comando.

Dessa forma, não é necessário alugar um servidor ou utilizar o navegador para executá-lo.

Esse tipo de uso é ideal para scripts executados usando o **cron** (*Unix, Linux*) ou o **Agendador de Tarefas** (*no Windows*). Esses scripts podem ser usados também para rotinas de processamento de texto simples.

Apesar das ótimas características, das grandes empresas que utilizam e das incríveis e vastas coisas que são possíveis de se fazer com essa linguagem, existe uma pergunta famosa que vem sempre à tona, “O **PHP** está morrendo?”

MÓDULO I

É o fim da linguagem PHP?

Hoje em dia, há um intenso debate sobre o **PHP** estar morrendo ou não. Isso se deve à crescente popularidade de outras linguagens adequadas para o lado do servidor, como **JavaScript** (*Node.JS*), **Python**, **Golang** e outras. Mas o **PHP** está realmente morrendo? A resposta é não!

Apesar de algumas pessoas criticarem e fazerem memes alegando que o **PHP** está morrendo, o **PHP** ainda é utilizado, como dito anteriormente, para executar os servidores de quase **79%** de todos os sites da internet, isso quer dizer que a cada **10** sites que você visitou, existe uma chance de **8** deles estarem utilizando **PHP**.

MÓDULO I



Em termos de vagas de trabalho, o **PHP** é melhor do que muitas outras linguagens de programação, o que reforça a tese de que o **PHP** não será descartado. Os dados nos mostram que o **PHP** é uma linguagem que ainda é muito utilizada, que está em constante atualização e evolução e que ainda vai viver muito.

Portanto, fique tranquilo!

MÓDULO I

Conclusão

PHP é uma linguagem interpretada do tipo **server-side** com diversos propósitos. Porém, ela é principalmente utilizada para gerar conteúdos dinâmicos em um site.

Não é por acaso que o **PHP** é uma das mais utilizadas e populares linguagens de programação, isso se deve à sua natureza de código aberto e suas funcionalidades versáteis. Ela é simples o suficiente para novatos, o que a torna uma linguagem democrática que não restringe seu aprendizado apenas a profissionais da área de tecnologia. A natureza de código aberto incentiva e fortalece o crescimento de uma comunidade ativa, que colabora com o constante aperfeiçoamento da linguagem, o que acontece desde sua criação.

MÓDULO I

O código **PHP** está escrito entre as tags `<?php ?>`

```
1 <!DOCTYPE html>
2 <html>
3   <body>
4     <h1>Estamos aprendendo PHP!</h1>
5     <?php
6       echo "Vamos prosseguir aprendendo PHP";
7     ?>
8   </body>
9 </html>
```

Exemplo de uso do **PHP** (*index.php*)

O arquivo **index** foi salvo com a extensão **.php** para mostrarmos ao nosso interpretador que há um código **PHP** a ser interpretado. Além disso, no exemplo usamos a função **echo** para escrever na tela uma mensagem.

MÓDULO I

Como comentar o código no PHP

Para comentarmos o nosso código **PHP** usamos duas barras ou # para comentários de uma linha, e para comentários de múltiplas linhas usamos /* */, o mesmo usado no **CSS**.

```
1  <?php
2      echo "Oi, Eu serei visto na sua tela";
3      // Eu não! Sou apenas um comentário.
4
5      echo "Oi, Eu também serei visto por você";
6      # Já eu não serei!
7
8      echo "E eu aqui novamente na sua tela, rs";
9      /* Eu não aparecerei na sua tela novamente
10     pois sou um comentário */
11  ?>
```

Exemplo de comentários no **PHP** utilizando //, # ou /**/

MÓDULO I

Variáveis no PHP

Para criarmos uma variável basta utilizar o sinal de **cifrão**. Uma variável pode armazenar textos e números. Além disso, a linguagem **PHP** é case sensitive, então **A** é diferente de **a**.

```
1  <?php
2      $name = "Guilherme";
3      $age = 20;
4
5      echo $name; // Guilherme
6      echo "</br>";
7      echo $age; // 20
8  ?>
```

MÓDULO I

No exemplo criamos uma variável **\$name** e declaramos a ela uma **string**, sendo assim precisamos colocá-la entre **aspas**. Já a outra variável **\$age** é declarada como **inteiro**, então não é necessário o uso de **aspas**. Ao usarmos o comando **echo** nas variáveis, o resultado impresso é o conteúdo dessa variável.

Para a nomeação de variáveis, as dicas a seguir são necessárias:

- **Não** inicie o nome de uma variável com **números**;
- **Não** utilize espaços em brancos;
- **Não** utilize caracteres especiais, somente **underline**;
- **Crie** variáveis com nomes que ajudarão a identificar melhor a mesma;
- **Evite** utilizar letras **maiúsculas**.

MÓDULO I

Falaremos agora sobre alguns dos tipos de variáveis que existem no **PHP**:

- **Booleanos**: Este é o tipo mais simples, pois só pode expressar apenas dois valores: **TRUE (1)** ou **FALSE (0, null ou uma string vazia)**;
- **Integer**: é um número **inteiro**, podendo ser **negativo** ou **positivo**;
- **Float**: também chamado de **double** ou **números reais** representados com um ponto para separar os dígitos do valor **inteiro** dos dígitos do valor das **casas decimais**.
- **Strings**: é uma **palavra** ou **frase** entre **aspas simples** ou **duplas**, assim como também pode ser **binário**, como o conteúdo de um arquivo **MP3** ou **JPG**.

MÓDULO I

Exemplos de String

```
1 <?php
2   $a = "mundo!";
3   echo "Olá, $a"; // Olá, mundo!
4   echo 'Olá, $a'; // Olá, $a
5   ?>
```

Note que quando declaramos no echo "Olá, \$a";, o **PHP** interpretou o conteúdo da variável **\$a**, pois está entre **aspas duplas**. E quando usamos a mesma forma, só que entre **aspas simples** echo 'Olá, \$a';, não temos o mesmo resultado. Então quando queremos que o **PHP** interprete o valor de nossa variável dentro de uma **string** é necessário o uso de **aspas duplas**.

Além disso, podemos usar um **ponto** para **concatenar strings**, assim como o sinal + para o **JavaScript**

```
1 <?php
2   echo "Olá," . " mundo!";
3   //Olá, mundo!
4   ?>
```


MÓDULO I

Constantes no PHP

O valor de uma constante jamais poderá ser alterado enquanto estiver sendo executada e para defini-la utilizamos a função ***define()*** ou ***const***

```
1  <?php
2      define("PHP", "Linguagem Open - Source");
3
4      const HTML = "Linguagem de marcação";
5
6      echo PHP; // Linguagem Open - Source
7
8      echo HTML; // Linguagem de marcação
9  ?>
```

Exemplo de Constantes

Utilizando a função ***define()*** ou a palavra reservada ***const*** definimos que as **constantes** com os nomes de **PHP** e **HTML** respectivamente, com o valor: **Linguagem Open – Source** e **Linguagem de marcação**.

MÓDULO I

Arrays no PHP

Um **array** que mantém uma série de elementos que podem ter diferentes tipos.

```
1  <?php
2      $php = array("Zend" => "CERTIFICAÇÃO", 6 => false);
3      echo $php["Zend"]; // CERTIFICAÇÃO
4      echo $php[6]; // 0
5
6      // Zend é nossa chave e CERTIFICAÇÃO nosso valor
7      // 6 é nossa chave e false(0) é nosso valor
8  ?>
```

Exemplo de arrays

Note que nossa primeira chave se chama **Zend**, e a outra chama-se **6**, mas quanto a nomeação de chaves de **array** pode ser tanto **string** ou um **integer**. Para o **valor** pode ser qualquer coisa.

MÓDULO I

Conversão de tipos

Os tipos de **variáveis** no **PHP** são **dinâmicos**. Para forçarmos os tipos de nossas **variáveis** utilizamos uma técnica conhecida como **type casting**, ou simplesmente **troca de tipos**.

```
1  <?php
2      $var = 100;
3      $type_casting = (bool) $var; // torna - se booleano
4      $type_casting = (int) $var; // torna - se inteiro
5      $type_casting = (float) $var; // torna - se float
6      $type_casting = (string) $var; // torna - se string
7      $type_casting = (array) $var; // torna - se array
8      echo $type_casting = (bool)$var; // 1
9  ?>
```

Exemplo de conversão de tipos

Veja que transformamos o valor da **\$var**, que antes era um **inteiro**, para um valor **booleano**.
Vamos conhecer agora **operadores**, que permitem que nós manipulemos o conteúdo de uma ou mais **variáveis**.

MÓDULO I

Operadores Aritméticos no PHP

Podemos utilizar operadores matemáticos para efetuar cálculos com os valores de **variáveis**

```
1  <?php
2    $a = 3;
3    $b = 3;
4    $c = $a * $b; // resultado é 9
5    $d = $a + $b; // resultado é 6
6    $e = $c - $d; // resultado é 3
7  ?>
```

Exemplo de operadores aritméticos

Criamos as variáveis **\$a** e **\$b** e a partir delas conseguimos fazer vários cálculos matemáticos.

MÓDULO I

Os operadores matemáticos disponíveis em **PHP** são:

- Adição: +
- Subtração: -
- Multiplicação: *
- Divisão: /
- Módulo: %

Lembrando que não precisamos especificar os tipos de variáveis no **PHP**, como nos exemplos a seguir:

```
1  <?php
2      $a = "5"; // string
3      echo $a + 2; // 7, integer
4      echo $a + '5 carros'; // 10, integer
5  ?>
```

MÓDULO I

Note que **\$a** é uma **string** e quando demos um **echo** nela somando com **2**, que é um **inteiro**, o resultado retornado foi **7**. Isso demonstra que nossos tipos de **variáveis** em **PHP** são sempre **dinâmicos**.

E na linha **4** o **PHP** reconhece dentro das **aspas simples** um **numeral** e com isso executa a operação matemática e exibirá o resultado **10** do tipo **inteiro** mas não exibindo a palavra "**carros**".

MÓDULO I

Operadores de Atribuição no PHP

Utilizamos os operadores de **atribuição** para definir **variáveis** e seus valores, além de usá-los juntamente com os operadores matemáticos

```
1  <?php
2  $a = 1; // A variável $a é igual a 1
3  $a += 2; // Somamos 2 ao valor da $a;
4  echo $a;
5  ?>
```

Exemplo de Operadores de Atribuição com Operadores Matemáticos

O resultado acima é **3**, pois somamos **2** ao valor da **\$a**, que é **1**

MÓDULO I

```
1  <?php
2      $a -= 2; // Subtraímos 2 ao valor da variável $a;
3      $a *= 2; // Multiplicamos o valor da variável $a por 2;
4      $a /= 2; // Dividimos o valor da variável $a por 2.
5  ?>
```

Exemplo de Operadores de Atribuição com Operadores Matemáticos

A **sintaxe** desses operadores é a mesma do exemplo da **soma**, pois basta dar um **echo** depois de ter declarado a **variável** com seu respectivo **operador**.

MÓDULO I

Podemos também incrementar ou decrementar variáveis utilizando os operadores de incremento, herdados da **linguagem C**.

```
1  <?php
2    $a = 1;
3    echo ++$a; // Incrementamos 1 e retornamos o valor
4    echo $a++; // Retornamos o valor e incrementamos 1
5    echo --$a; // Decrementamos 1 e retornamos o valor
6    echo $a--; // Retornamos o valor e decrementamos 1
7  ?>
```

Exemplo de Operadores de Atribuição - Incremento e decremento

MÓDULO I

Operadores Relacionais

Esses são usados para comparar valores ou expressões, retornando um valor booleano (*true ou false*):

- Igual: ==
- Idêntico: ===
- Diferente: != ou <>
- Menor que: <
- Maior que: >
- Menor ou igual: <=
- Maior ou igual: >=

É importante lembrar que == **não** checa o tipo da **variável**, apenas seu **valor**.
Já o === checa tanto o valor da **variável** quanto o seu **tipo**.

MÓDULO I

Operadores Lógicos

Existem também os operadores lógicos para a criação de testes condicionais:

- `$a and $b`: enquanto A e B forem verdadeiros;
- `$a or $b`: enquanto A ou B forem verdadeiros;
- `$a xor $b`: enquanto A ou B forem verdadeiros, mas não os dois;
- `!$a`: verdadeiro se A for falso;
- `$a && $b`: enquanto A e B forem verdadeiros;
- `$a || $b`: enquanto A ou B forem verdadeiros.

MÓDULO I

Estrutura de Decisão if/else

A condição é avaliada para que, caso algo seja **verdadeiro**, **faça isto**, senão, **faça aquilo**.

```
1  <?php
2      $idade = 17;
3
4      if($idade < 18) {
5          echo 'Você não pode entrar aqui!';
6      } else {
7          echo 'Seja bem - vindo';
8      }
9  ?>
```

Exemplo do uso das estruturas de decisão **if/else**

MÓDULO I

- Na linha **1** criamos a variável **\$idade** que guarda um **inteiro**.
- Na linha **4** utilizamos **IF** para verificar se **\$idade** é menor que **18**, e caso seja será impresso: *"Você não pode entrar aqui!"*
- Na linha **6** criamos um **ELSE**, que é o contrário da primeira condição.

Podemos também utilizar os **operadores lógicos** junto dos **operadores relacionais**.

```
1  <?php
2  $idade = 21;
3  $identidade = true;
4
5  if($idade > 18 && $identidade == true) {
6      echo 'Seja bem-vindo!';
7  }
8  ?>
```

Exemplo do uso de Operadores lógicos com Operadores relacionais

MÓDULO I

Estruturas de Decisão (else if/switch)

Podemos utilizar a estrutura **ELSE IF** quando criamos uma outra condição, além da principal.

```
1  <?php
2  $nome = 'Till Lindemann';
3
4  if($nome == 'Richard Kruspe') {
5      echo 'E ae Richard Kruspe!';
6  } elseif ($nome == 'Oliver Riedel') {
7      echo 'E ae Oliver Riedel!';
8  } elseif ($nome == 'Till Lindemann') {
9      echo 'E ae Till Lindemann!';
10 } else {
11     echo "E ae $nome!";
12 }
13 ?>
```

MÓDULO I

Declaramos para **\$nome** uma **string** e depois fizemos várias condições, onde caso não caia em nenhum **elseif**, será retornado o valor que estiver dentro da **\$nome**.

O **ELSE IF** pode ser muito útil, mas o mesmo é aconselhável usar apenas quando temos poucas condições. Caso contrário, para não manter um código cheio de **ELSE IF's**, o mais indicado é usar o **SWITCH**, que permite criarmos infinitas condições de forma organizada.

MÓDULO I

```
1 <?php
2     $nome = 'Fulano';
3
4     switch($nome) {
5         case 'Fulano':
6             echo 'E ai Fulano!';
7             break;
8
9         case 'Sicrano':
10            echo 'E ai Sicrano!';
11            break;
12
13        case 'Beltrano':
14            echo 'E ai Beltrano!';
15            break;
16
17        default:
18            echo 'Qual é o seu nome?';
19            break;
20    }
21
22    // Resultado é: E ai Fulano!
23    ?>
```

No exemplo foi criada **\$nome** declarando a ela uma string. É verificado a variável passada em **switch** entre parênteses: caso o valor contido na variável seja o que estiver em "**case**", será impresso o que conter no **echo** já pré-definido, e o **break** para a verificação. Caso não seja na primeira condição, será verificado todas as demais condições até chegar na última, que perguntará o seu nome. Na sintaxe básica do **Switch** atente-se sempre ao ":" no case e os ";" depois do **echo** e **break**.

MÓDULO I

Operador Ternário no PHP

No **PHP** existe uma forma mais curta de criar condições através do **Operador Ternário**

```
1  <?php
2      $number1 = 1;
3      $number2 = 2;
4
5      if($number2 > $number1) {
6          $a = 'Número 2 é maior que número 1';
7      } else {
8          $b = 'Número 2 não é maior que número 1';
9      }
10
11     $ternario = ($number2 > $number1) ?
12     'Número 2 é maior que número 1' : 'Número 2 não é maior que número 1';
13
14     echo $ternario; // Número 2 é maior que número 1
15  ?>
```

MÓDULO I

Criamos duas variáveis e a partir delas fizemos verificação com **if/else** e da forma ternária também.

Na **\$ternario** passamos a condição que tem o mesmo valor que o **if** entre parênteses e o ponto de interrogação faz a pergunta: **\$number2** é maior **\$number1**?

Caso seja, será impresso o que vier depois desse ponto. Caso **\$number2** não seja maior que **\$number1**, será impresso o que estiver depois dos “:” que têm o mesmo valor que **else**. Sendo assim, podemos comparar qual forma é mais simples com esses dois exemplos.

MÓDULO I

Arrays Associativos

Quando criamos um **array**, por padrão, ele recebe **chaves** numéricas incrementadas automaticamente de acordo com novos valores. Contudo, podemos criar **chaves** que são **strings**, daí chamamos de **array associativo**. Para explicar melhor, faremos um exemplo em que mostraremos as estações nos meses do ano.

```
1  <?php
2      $estacao = array('Verão' => 'de 21 de dezembro a 21 de março',
3                      'Outono' => 'de 21 de março a 21 de junho',
4                      'Inverno' => 'de 21 de junho a 23 de setembro',
5                      'Primavera' => 'de 23 de setembro a 21 de dezembro');
6  ?>
```

Arrays Associativos

MÓDULO I

Observem que nossos valores agora possuem nomes. Podemos utilizar estes nomes para nos referenciarmos a um valor específico dentro de um **array**.

Veja como fica na prática o exemplo:

```
1 <?php
2 $estacao = array('Verao' => 'de 21 de dezembro a 21 de março',
3   'Outono' => 'de 21 de março a 21 de junho',
4   'Inverno' => 'de 21 de junho a 23 de setembro',
5   'Primavera' => 'de 23 de setembro a 21 de dezembro');
6 echo "A estação Verão foi: {$estacao['Verao']}";
7
8 // A estação Verão foi: de 21 de dezembro a 21 de março
9 ?>
```

Como sabemos, a sintaxe de um **array** é chave => valor. Note que demos um **echo** assim: **{ \$estacao['Verao'] }**, onde **\$estacao** contém um **array** que tem uma chave chamada **Verao** e contém o valor: *de 21 de dezembro a 21 de março*. Sendo assim, será impresso o valor da chave especificada.

MÓDULO I

Existem maneiras diferentes de utilizar variáveis dentro de **strings** e vice-versa, concatenadas ou não concatenadas

```
1  <?php
2  $ensino = 'EAD';
3  $formacao = array('PHP' => 'Desenvolvedor PHP', 'Infra' => 'SysAdmin Linux');
4
5  // Não concatenadas
6  echo "<p>No $ensino da DevMedia você se torna {$formacao['PHP']}";
7  echo " e pode se tornar também {$formacao['Infra']}.</p>";
8
9  // Concatenadas
10 echo '<p>No ' . $ensino . ' da DevMedia você se torna ' . $formacao['PHP'];
11 echo ' e pode se tornar também ' . $formacao['Infra'] . '.</p>';
12 ?>
```

No exemplo o resultado será o mesmo, apenas mostramos a utilização de uma variável e um **array** com concatenação ou não concatenado para mostrar seus valores. Tente usar sempre a forma não concatenada: você pode notar que nosso código ficou bem mais limpo.

MÓDULO I

Arrays Multidimensionais

Arrays multidimensionais são, basicamente, array dentro de um array

```
1  <?php
2      $temp = array(
3          '2010' => array(
4              'Outubro' => 25,
5              'Novembro' => 23,
6              'Dezembro' => 20),
7          '2011' => array(
8              'Outubro' => 26,
9              'Novembro' => 22,
10             'Dezembro' => 21),
11          '2012' => array(
12              'Outubro' => 27,
13              'Novembro' => 28,
14              'Dezembro' => 19)
15      );
16
17      echo "Dezembro de 2012 foi de: {$temp['2012']['Dezembro']} graus";
18      // Dezembro de 2012 foi de: 19 graus
19  ?>
```

MÓDULO I

Veja que criamos a **\$temp** que guarda um array. Em nosso exemplo, o array com o nome de **2010** guarda os valores **Outubro**, **Novembro** e **Dezembro** em um outro array, que por sua vez, guarda outros valores, que são a temperatura. Veja que demos um **echo** em `{ $temp['2012']['Dezembro'] }` onde **\$temp** guarda um array **2012** e inicia um outro array que contém a chave **dezembro** e que tem o valor que queremos.

MÓDULO I

Criando Arrays de uma forma alternativa

Podemos criar **arrays** de uma forma simples e rápida utilizando o operador []

```
1  <?php
2      $number1 = array(100, 101, 102);
3      $number1[] = 103;
4      $number2[] = 104;
5
6      print_r($number1);
7      echo '<hr/>';
8      print_r($number2);
9  ?>
```

Criando **Arrays** de forma alternativa

MÓDULO I

Através dos **colchetes** conseguimos acrescentar dados a um **array**.

No caso da **\$number1** temos um **array** com os valores **100, 101, 102**.

Quando declaramos **\$number1[] = 103**; estamos dizendo que queremos acrescentar no **array \$number1** o valor **103**.

Podemos também alterar os valores de um **array** usando **colchetes**

```
1  <?php
2    $cert = array('EAD' => 'Você terá um certificado ', 'PHP' => 'Linux');
3
4    $cert['PHP'] = 'Zend';
5    print_r($cert);
6  ?>
```

A sintaxe básica do **DO WHILE** é primeiro executar o que colocamos entre as chaves do **DO**.

MÓDULO I

FOR

O **for** é igual ao **while** e ao **do while**, permitindo que executemos três operações em sua condição, separadas por ponto e vírgula.

A **primeira** é executada ao início do **loop**, a **segunda** é a condição (*enquanto ela for verdadeira, o loop continuará*), e a **terceira** é executada ao fim de cada repetição

```
1  <?php
2  for($a = 1; $a <= 10; $a++){
3      $cubo = $a * $a * $a;
4      echo "O cubo de $a é $cubo<br />";
5  }
6  ?>
```

Entre parênteses temos **\$a** que guarda **1**. Depois verificamos se **\$a** for menor ou igual a **10** e, por último, incrementamos **\$a**. Dentro das chaves do **FOR** criamos uma variável que traz o cubo de **\$a** para nós e quando demos um **echo** será mostrado o cubo dos números de **1** a **10**.

MÓDULO I

Foreach

O **Foreach** faz o mesmo que as demais estruturas já apresentadas, porém, com ela podemos trabalhar com **arrays**

```
1  <?php
2      $ead = array('Aqui na DevMedia ', 'você se torna um ', 'desenvolvedor PHP');
3
4      foreach($ead as $can){
5          echo "$can";
6      }
7
8      //Aqui na DevMedia você se torna um desenvolvedor PHP
9  ?>
```

No exemplo é criado um **array** e depois usamos o **foreach** para ir nesse **array** e repetir tudo o que conter nele. A sintaxe do **foreach** é mostrada entre parênteses onde colocamos o nome de nossa variável, e com o termo as alteramos o nome dela para **\$can**. Depois, basta dar um **echo** que tudo que conter em nosso array será mostrado.

MÓDULO I

Utilizamos o loop em um **array** que contém chave => valor.

```
1  <?php
2  $temp = array('Outubro' => 27, 'Novembro' => 28, 'Dezembro' => 19);
3
4  foreach($temp as $chave => $valor) {
5      echo "A temperatura média de $chave foi de $valor graus<br />";
6  }
7  ?>
```

Quando formos utilizar o **foreach** em um array que contém chave e valor é necessário a sintaxe que vemos entre parênteses.

MÓDULO I

```
1  <?php
2  $temp = array(
3      '2010' => array(
4          'Outubro' => 25,
5          'Novembro' => 23,
6          'Dezembro' => 20),
7      '2011' => array(
8          'Outubro' => 26,
9          'Novembro' => 22,
10         'Dezembro' => 21),
11      '2012' => array(
12          'Outubro' => 27,
13          'Novembro' => 28,
14          'Dezembro' => 19)
15  );
16
17  foreach($temp as $ano => $meses){
18      echo "Temperaturas em $ano<br />";
19
20      foreach($meses as $mes => $temp) {
21          echo "$mes: $temp graus<br />";
22      }
23  }
24  ?>
```

Assim como temos os arrays multidimensionais, assim também temos **foreachs** multidimensionais, que são **foreachs** dentro de **foreachs**, como mostra o exemplo.

Veja que criamos um array e dentro dele vários outros arrays que guardam temperaturas de meses de um determinado ano. Quando utilizamos o **foreach** mostraremos o ano da temperatura e quantos graus no mês especificado.

MÓDULO I

Quebrando loops

Enquanto estamos dentro de um **loop**, podemos utilizar duas instruções: **continue** e **break**. Elas permitem que nós quebrems os laços de repetição.

Note que não aparece o cubo de **3**, pois fizemos a seguinte verificação: **if(\$a == 3) {continue;}**.

```
1  <?php
2  for($a = 1; $a <= 10; $a++){
3      if($a == 3) {
4          continue;
5      }
6
7      $cubo = $a * $a * $a;
8      echo "O cubo de $a é $cubo<br />";
9  }
10
11  //O cubo de 1 é 1
12  //O cubo de 2 é 8
13  //O cubo de 4 é 64
14  //O cubo de 5 é 125
15  //O cubo de 6 é 216
16  //O cubo de 7 é 343
17  //O cubo de 8 é 512
18  //O cubo de 9 é 729
19  //O cubo de 10 é 1000
20  ?>
```

MÓDULO I

Vemos um exemplo da instrução **Break**.

```
1  <?php
2  for($a = 1; $a <= 10; $a++){
3      if($a == 3) {
4          break;
5      }
6
7      $cubo = $a * $a * $a;
8      echo "O cubo de $a é $cubo<br />";
9  }
10 //O cubo de 1 é 1
11 //O cubo de 2 é 8
12 ?>
```

O **break** para o **loop**, então veja que só é mostrado o cubo de 1 e 2. Isso porque colocamos a verificação **if(\$a == 3) {break;}**, ou seja, chegou no 3 e deu um **break**.

MÓDULO I

EXERCÍCIOS...

MÓDULO I

FIM...